Aventure
partie 8

Remarque

— pour chaque classe référez vous à la javadoc
— limitez vous à ce que vous avez vu en cours
— méthode toString() à faire systématiquement

1 Les conditions de fin

1.1 Création des classes

Créez la classe `ConditionDeFin`, c'est une classe abstraite qui permettra de vérifier si une condition est remplie.

— Le constructeur stocke l'état du jeu si la condition était vérifiée
— La méthode `getEtatSiConditionVerifiee`, retourne cet état
— La méthode `verifierCondition`, vérifie la condition et retourne cet état si elle est vérifiée, sinon elle retourne `ENCOURS`

Le diagramme de classe vous montre les classes conditions de fin à créer, excepté `ConjonctionConditionDe`

Exemple :

Pour comprendre le fonctionnement, ces 2 instanciations permettent de vérifier si le jeu est terminé et selon le joueur décédé (RIP) il s'agira d'un `SUCCES` ou d'un `ECHEC`.

```
new ConditionDeFinVivantMort(EtatDuJeu.SUCCES, delestre);
new ConditionDeFinVivantMort(EtatDuJeu.ECHEC, malandain);
```

1.2 Modification des `Lecteur` et `Enregistreur`

Modifiez votre code afin de prendre en compte des collections de conditions de fin, regardez l'API et le diagramme de classes.

Facultatif

Si vous avez terminez tout le TP, prenez le temps pour généraliser les conjonctions de conditions de fin en créant la classe `ConjonctionConditionDeFin`, dont le constructeur a la signature suivante :
`ConjonctionConditionDeFin(EtatDuJeu etat, ConditionDeFin... conditions)`

Recoder votre classe `ConditionDeFinVivantDansPieceEtPossedeObjets`.

2 La méthode `executer` de `JoueurHumain`

Cette méthode va disséquer la chaîne correspondant à l'ordre stocké dans `JoueurHumain`. Exemples d'ordre :

- OuvrirPorte Porte42
- OuvrirPorte Porte666 avec cledudiable
- Franchir Porte42

Aide à la réalisation

- le 1er token donne l'ordre, le concaténer à "commande" pour avoir le nom de la méthode à rechercher
- le reste des tokens contient les paramètres effectifs de la méthode
  - construisez un tableau de `String` avec les paramètres effectifs
  - construisez un tableau de `Class` de même taille rempli de `String.class` `Arrays.fill` peut vous aider.
- recherchez la méthode avec son nom et les types des paramètres
- invokez la méthode avec les paramètres effectifs

Vous devrez traiter dans l'ordre 2 types d'exceptions dans cette méthode :

1. `InvocationTargetException`, elle est lancée si l'invocation de la méthode déclenche des exceptions (ll s'agit des exceptions de `ITIAventureException`), le traitement va consister à récuperer et relancer la cause (l'exception initiale) qui sera donc propagée.
2. `Exception`, tout autre exception sera lié à un problème lié à l'absence de méthode correspondante. Lancez l'exception `CommandeImpossiblePourLeVivantException` avec un message indiquant l'impossibilité de l'ordre.

Le traitement des exceptions dans le simulateur sera d'afficher le message afin de prévenir le joueur.

3 La classe `Simulateur`

Le simulateur va permettre :
— soit de jouer 1 tour et vérifier les conditions
— soit de jouer jusqu'à ce qu'une condition de fin soit vérifiée (différente de `ENCOURS`)
Le principe d'`executerUnTour` est le suivant :
— vérifiez que l'état du jeu est bien `ENCOURS` sinon sortir
— affichez la situation du joueur lui permettant de voir :
  — ce qu'il porte et ses points
  — sa pièce avec les objets, les vivants et les portes avec leur état
  — *normalement vous avez des `toString`, si il faut modifiez les*
— demandez à chaque joueur humain son ordre
— exécutez tous les `Executable`
— vérifiez les conditions et retournez le nouvel état du jeu (le stocker en attribut peut être utile)

Facultatif

Si vous avez terminez tout le TP, prenez le temps de généraliser, vous avez normalement fait des méthodes privées `getExecutables` et `getJoueurHumains`. Le code dans ces 2 méthodes est le même. Développez une méthode générique ayant cette signature :

```
private <T> Collection<T> getEntites(Class<T> classe)
```

Utilisez les stream d'objet à l'aide de `filter` et `map` pour obtenir les collections d'entités dont vous avez besoin. Les méthodes de la classe `Class` vous seront utiles dans le `filter` et le `map`.

4 `Main`

Le code source de la classe `Main` vous est fourni dans sa totalité.

À vous de jouer maintenant !!!

Attention, le PI qui suivra dans quelques semaines utilisera le jeu d'aventure que vous venez de coder. Il **doit** d'être fonctionnel.